CORSO AVANZATO DI INFORMATICA E ROBOTICA

LEZIONE 12: Python

Carlo **Zambaldo** | 11 febbraio 2026

Gestione I/O in Python

01

Esempio – inserire dati da command window

La funzione input() permette di leggere dati inseriti dall'utente da tastiera.

- Il programma si ferma
- Aspetta che l'utente scriva qualcosa
- Quando premi INVIO, il valore viene salvato nella variabile

```
>>> a = input()
10
>>> b = input()
22
>>> a+b
'1022'
>>> |
```

```
num = input("inserire un numero intero: ")
print(f"Hai inserito: {num}")
```

input() restituisce sempre una **stringa** (testo)!!

```
>>> int(a)+int(b)
32
>>> |
```

Convertire i dati

```
eta = int(input("Quanti anni hai? "))
anno_prossimo = eta + 1
print("L'anno prossimo avrai", anno_prossimo)
```

Conversioni più comuni:

int() → numero intero

float() → numero con virgola

str() → testo

```
numero = float(input("Inserisci un numero: "))
if numero > 0:
        print("Numero positivo")
else:
        print("Numero negativo o zero")
```

Gestione File

Perché leggere un file?

- Salvare dati in modo permanente
- Riutilizzare informazioni (configurazioni, testi, numeri)
- Comunicare tra programmi

Per aprire un file:

```
file = open("dati.txt", "r")
```

nota: la «r» sta per read («a» = append, «w» = write)

Per leggere il contenuto:

```
contenuto = file.read()
print(contenuto)
```

Per chiudere il file

```
file.close()
```

È consigliato usare «with» per aprire il file, perché:

- il file si chiude automaticamente
- codice più pulito
- meno errori

```
with open("dati.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
```

Gestione File

Perché leggere un file?

- Salvare dati in modo permanente
- Riutilizzare informazioni
- Comunicare tra programmi

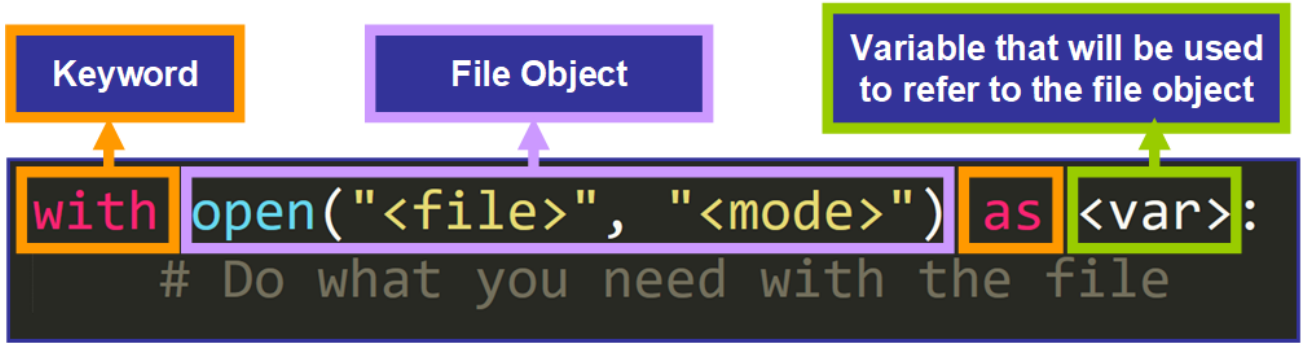
Le modalità disponibili sono:

- **Lettura (`"r"`, read)**
- **Aggiunta (`"a"`, append)**
- **Scrittura (`"w"`, write)**
- **Creazione (`"x"`, create)**

Puoi anche scegliere di aprire un file in:

- **Modalità di testo (`"t"`)**
- **Modalità binaria (`"b"`)**

https://www.freecodecamp.org/italian/news/i-file-in-python-aprire-leggere-scrivere-file-e-altre-funzioni-di-gestione-file/



È consigliato usare «with» per aprire il file, perché:

- il file si chiude automaticamente
- codice più pulito
- meno errori

```
with open("dati.txt", "r") as file:
        contenuto = file.read()
        print(contenuto)
```

Esercizio

1. Creare un file .txt usando «nano», scrivere quindi almeno due righe di testo
2. Aprire un file .py, scrivere il codice:

```
with open("dati.txt", "r") as file:
    contenuto = file.read()
print(contenuto)
```

3. Ora, provare ad eseguire
4. Cosa cambia se il codice è:

```
with open("dati.txt", "r") as file:
    for riga in file:
        print(riga, end="")
```

Scrivere su file

fondiamo insieme basic I/O con scrittura su file!

```
messaggio = input("Scrivi qualcosa per il diario: ")

with open("diario.txt", "a") as file:
    file.write(messaggio + "\n")

print("\nContenuto del diario:\n")

with open("diario.txt", "r") as file:
    for riga in file:
            print(riga, end="")
```

```
Scrivi qualcosa per il diario: ciaoooo

Contenuto del diario:

ciaoooo


** Process exited - Return Code: 0 **
```

Librerie Python e Ambienti Virtuali

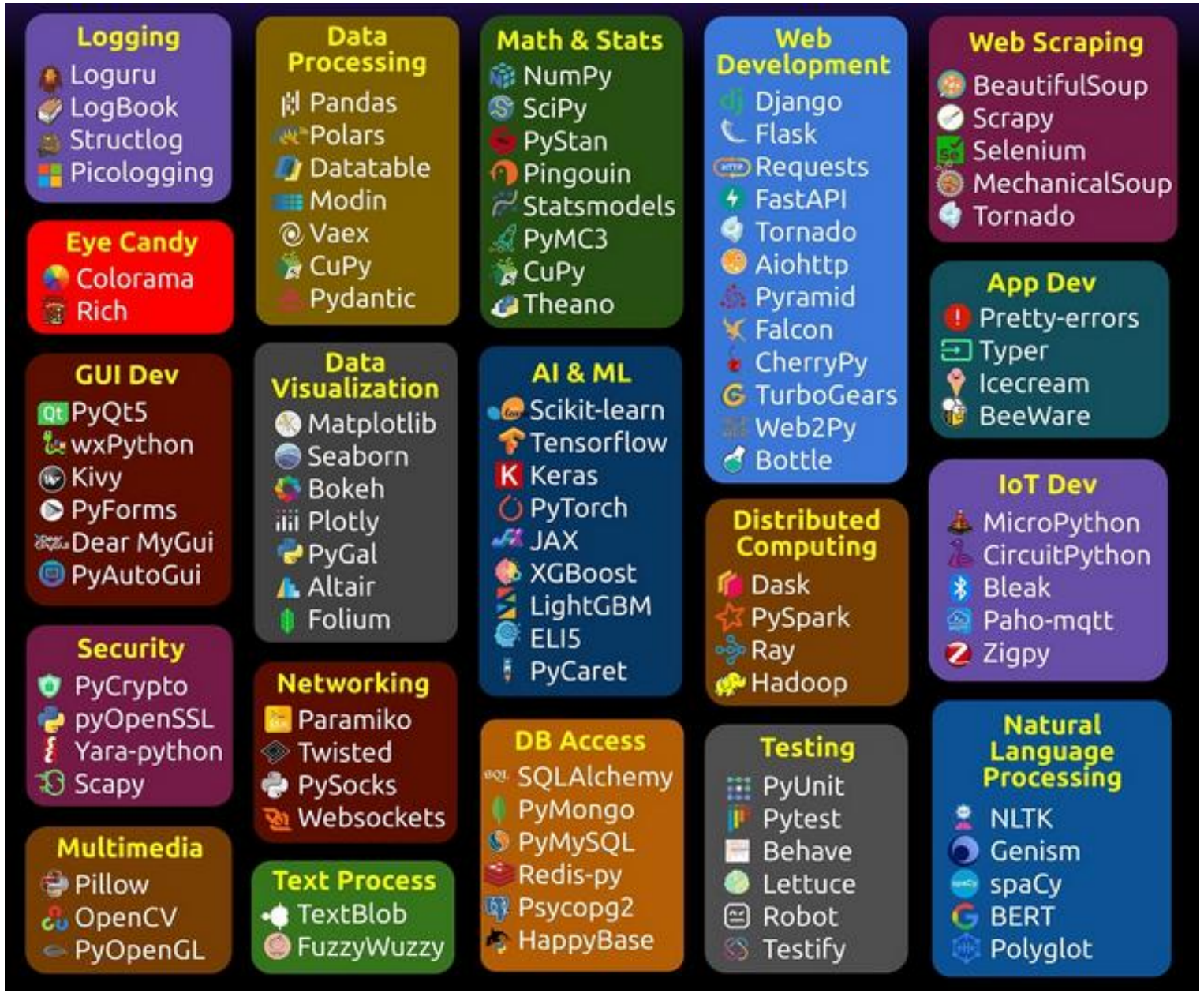
03

Librerie Python di terze parti

Uno dei principali punti di forza del linguaggio Python è il suo vasto ecosistema di librerie.

Sebbene le librerie integrate in Python siano già molto versatili, esistono moltissime librerie e framework di terze parti che rendono la programmazione in Python ancora più produttiva.

Ecco alcune delle librerie e dei framework open-source Python più utili e popolari.



Come importare una libreria / modulo

Arduino:

```
#include "esp_camera.h"
#include "img_converters.h"
#include "Arduino.h"
```

- **Import semplice**

```
import math
print(math.sqrt(16))
```

- **Import con alias**

```
import math as m
print(m.sqrt(25))
```

- **Importare solo una funzione**

```
from math import sqrt
print(sqrt(36))
```

Scrittura	Quando usarla
`import modulo`	più chiaro
`import modulo as x`	abbreviare
`from modulo import funzione`	usare poche funzioni

“Moduli”, “pacchetti”, “librerie”: cosa stai installando davvero?

- **Modulo**: un singolo file .py (es. mio_modulo.py).
- **Pacchetto**: una cartella con codice Python (spesso con __init__.py) e magari sotto-moduli.
- **Libreria**: termine generico (spesso coincide con un pacchetto installabile).

`pip` è il **package manager ufficiale di Python**.

Un *package manager* è un programma che:

- scarica librerie
- le installa
- gestisce le versioni
- risolve le dipendenze

```
pip install <nome_pacchetto>
```

Cosa fa pip:

1. si collega a **PyPI (Python Package Index)**
2. scarica il pacchetto **`<nome_pacchetto>`**
3. scarica automaticamente le sue dipendenze
4. installa tutto dentro il tuo ambiente Python

https://pypi.org/

Dove viene installato un modulo?

Il problema: su un computer possono esistere **più versioni di Python**:

- Python di sistema
- Python installato manualmente
- Python dentro un venv

Usando `pip install nome_pacchetto` non è sempre chiaro **quale Python** sta ricevendo il pacchetto, e si possono ottenere errori tipo: ModuleNotFoundError

È conveniente usare: `python3 -m pip install nome_pacchetto`
Questo dice al computer: «Usa pip appartenente a questo Python.»

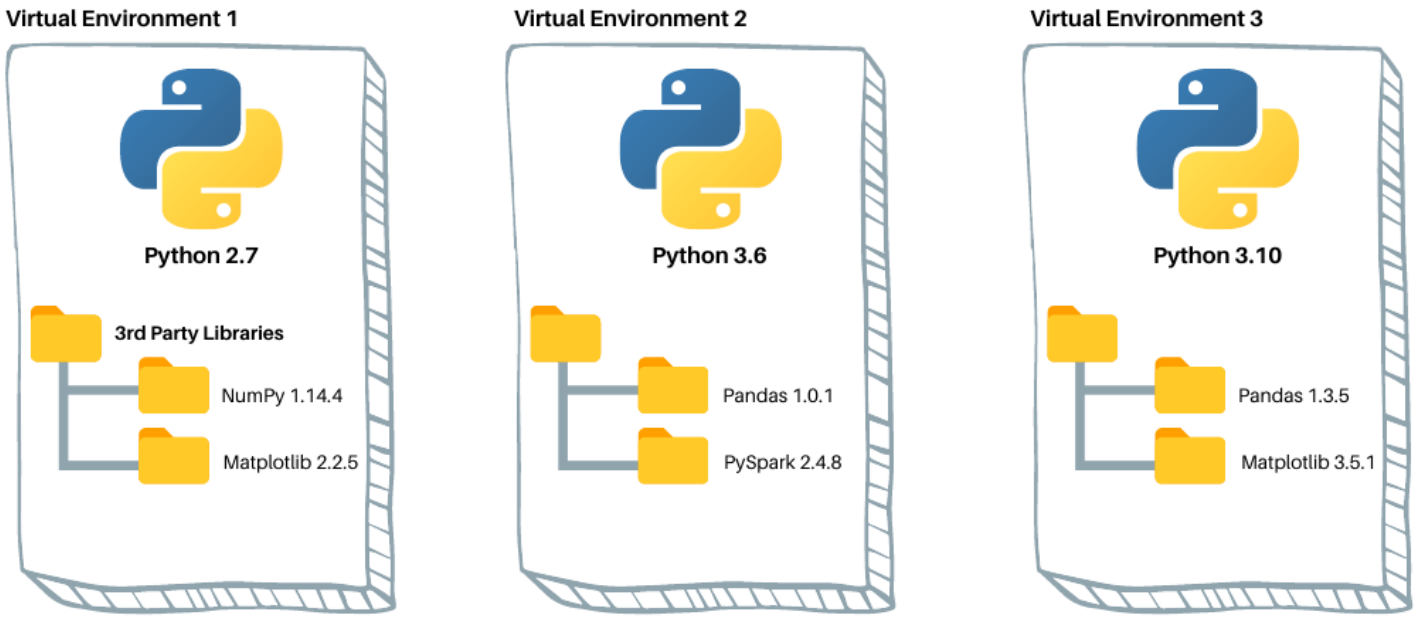
Il problema non è installare un modulo.
Il problema è installarlo nel Python giusto.

Un metodo ancora più sicuro è l'utilizzo degli ambienti virtuali

Ambienti Virtuali
Perché usare venv è conveniente?

Se installi tutto nel Python di sistema:

- Versioni che entrano in conflitto
- Progetti che smettono di funzionare
- Ambiente “sporco”
- Difficile replicare su un altro PC



venv crea un ambiente Python isolato.
Ogni progetto ha:

- **il suo Python**
- **le sue librerie**
- **le sue versioni**

Con installazione globale → possibili conflitti!

Benefici:

- **Nessun conflitto tra progetti**
- **Ambiente replicabile** (`pip freeze > requirements.txt` → ricrearli altrove con `pip install -r requirements.txt`)

Accenni sui Container

venv	Container (es. Docker / Podman)
Isola **solo Python e le sue librerie**	Isola: • Python • Librerie • Sistema operativo • Dipendenze di sistema • Configurazione
✔ Versioni di pacchetti indipendenti ✔ Leggero ✔ Facile da usare ❌ Non isola il sistema operativo ❌ Non isola librerie di sistema Funziona **dentro il tuo computer**.	È una **mini-macchina virtuale leggera**. ✔ Ambiente completamente replicabile ✔ Funziona uguale su ogni macchina ✔ Ideale per server / produzione ❌ Più complesso ❌ Più pesante di un venv

Domande, Dubbi, Perplessità?